ULTIMATE C PROGRAMMING HANDBOOK

CODEWITHHARRY



Introduction to Programming & C

My name is Harry and I am a YouTuber at CodeWithHarry. I have been teaching programming for years and I know how confusing it can feel when you are just starting out. That is why I thought of creating the best learning experience for the C language, designed specially for beginners. If you follow along, you will not only learn C, but also build the kind of programming foundation that will help you pick up any language in the future.

Why Learn Programming?

Programming is the art of telling computers what to do. Every piece of software you use-from your operating system to the apps on your phone-exists because someone wrote instructions for the computer to follow.

When you learn programming, you gain the ability to solve problems with code. You stop being a passive user of technology and become a creator. This skill is extremely valuable in today's world, not just for software developers, but also for anyone in science, engineering, business, or even creative fields.

Learning programming teaches you logical thinking, problem solving, and efficiency. These skills will serve you well beyond coding itself.

What is C Language? History & Importance

C is one of the oldest and most influential programming languages. It was developed in the early 1970s at Bell Labs by Dennis Ritchie. Despite being created decades ago, C is still widely used today.

CodeWithHarry

Why is that? Because C is:

- **Powerful**: It gives you low-level access to memory, making it great for system programming.
- **Efficient**: Programs written in C are fast and lightweight.
- **Foundational**: Many other languages (like C++, Java, and even Python's core) are built on top of concepts from C.

By learning C, you are learning the language that shaped modern programming.

Real-world Applications of C

C is not just a "beginner's language"-it is used in the real world for critical applications:

- Operating Systems (Linux, Windows, macOS all have components written in C)
- Embedded Systems (IoT devices, microcontrollers, consumer electronics)
- Game Engines (many early game engines were written in C)
- Databases (MySQL, Oracle)
- Compilers and Interpreters

Learning C means you can understand how software interacts with hardware at a deeper level.

How C Compares to Other Languages (C++, Python, Java)

- **C vs Python**: Python is easy to read and write, but slower. C is faster and closer to hardware.
- **C vs Java**: Java runs on a virtual machine (JVM), making it portable. C is compiled directly, making it faster but less portable.
- **C vs C++**: C++ is an extension of C with object-oriented programming features. Learning C first helps you understand the roots of C++.

CodeWithHary

Think of C as the “mother language” of modern programming. Once you know it, moving to Python, Java, or C++ becomes much easier.

Installing a Compiler

Before you write C programs, you need a **compiler**. A compiler is a program that converts the C code you write into machine code that your computer can understand.

There are many options:

- **GCC (GNU Compiler Collection)**: A popular, free, and open-source compiler available on Windows, Linux, and macOS.
- **Turbo C**: An older IDE, often used in schools, but not recommended today.
- **Code::Blocks or VS Code**: Modern IDEs where you can write, compile, and run your C code easily.

We will go step-by-step and set up GCC with a beginner-friendly editor so that you can run your first program without any confusion.

Writing & Running Your First C Program

Now that your environment is ready, let’s write the most basic C program-**Hello, World!**

```c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

CodeWithHarry

Explanation:

- `#include <stdio.h>` : This line tells the compiler to include the standard input-output library, which allows us to use `printf` .
- `int main() { ... }` : The main function is where every C program begins execution.
- `printf("Hello, World!\n");` : Prints the text to the screen.
- `return 0;` : Indicates that the program finished successfully.

When you run this program, you will see:

```
Hello, World!
```

This might look simple, but it's the first step into the world of programming. From here, we will keep building step by step until you can write complex programs on your own.

CodeWithHarry

Practice Set – Introduction to Programming & C

These exercises will help you practice the basics you learned in this section. Try to solve them on your own before looking at any solutions.

Exercise 1: Print Your Own Message

Write a C program that prints your **name** and **age** on separate lines.

Example Output:

```
My name is Prakash
I am 25 years old
```

Exercise 2: Print a Motivational Quote

Write a program that prints your favorite motivational quote.

Example Output:

```
"Success is the sum of small efforts repeated day in and day out."
```

Exercise 3: Modify the Hello World Program

Take the “Hello, World!” program we wrote earlier and change it so that it prints:

CodeWithHarry

```
Hello, C Learner!
Welcome to the world of programming.
```

Exercise 4: Experiment with `main()` Return Value

Write a program where you:

1. Print a simple message.
2. Change the return value of `main()` from `0` to another number (like `5`).
3. Observe what happens when you run the program.

Exercise 5: Multiple `printf` Statements

Write a program that uses **three separate `printf` statements** to print:

```
I am learning C programming.
It feels exciting.
This is just the beginning!
```

Instructor's Note: At this stage, focus only on writing, compiling, and running programs. Do not worry about input from the user yet—we will cover that in the next section.

CodeWithHarry

Basics of C Programming

In the previous section, we set up our environment and wrote our first C program. Now it is time to dive deeper into the basics of C programming. By the end of this section, you will clearly understand how a C program is structured, how to work with variables, how to take input and display output, and how to make your code readable with comments.

Structure of a C Program

A C program generally follows a well-defined structure. Here is the typical format:

```c
#include <stdio.h>   // Preprocessor directive

int main() {         // Main function where execution begins
    // Your code goes here
    return 0;        // End of program
}
```

CodeWithHarry

Breakdown:

- **Preprocessor Directive**: Lines starting with `#` (like `#include <stdio.h>`) tell the compiler to include external files before compilation.
- **main() Function**: The entry point of every C program. Execution always begins here.
- **Statements**: Instructions written inside `{ }`. Each statement ends with a semicolon `;`.
- **return 0;**: Returns control to the operating system, indicating successful program completion.

Variables and Data Types

A variable is like a container in memory that holds a value. In C, every variable must have a data type that defines the kind of data it can store.

Common Data Types

- **int**: Stores integers (whole numbers), e.g., `10`, `-25`.
- **float**: Stores real numbers with decimal points, e.g., `3.14`, `-2.5`.
- **double**: Similar to float, but with double precision (more accurate for decimals).
- **char**: Stores a single character, e.g., `'A'`, `'z'`.

Example Program:

```c
#include <stdio.h>

int main() {
    int age = 20;
    float height = 5.9;
    double pi = 3.14159265;
    char grade = 'A';

    printf("Age: %d\n", age);
    printf("Height: %.1f\n", height);
    printf("Value of pi: %lf\n", pi);
    printf("Grade: %c\n", grade);

    return 0;
}
```

CodeWithHarry

Keywords and Identifiers

- **Keywords**: Reserved words in C that have special meaning and cannot be used as variable names. Examples: `int`, `float`, `if`, `else`, `return`, `while`.

- **Identifiers**: Names you give to variables, functions, or other elements. They must follow rules:
  - Can only contain letters, digits, and underscores (`_`).
  - Cannot start with a digit.
  - Cannot be a keyword.
  - C is case-sensitive, so `Age` and `age` are different identifiers.

Input and Output in C

To interact with users, C provides two standard functions:

- **printf**: Used to display output.
- **scanf**: Used to take input from the user.

Example Program:

```c
#include <stdio.h>

int main() {
    int age;
    float salary;

    printf("Enter your age: ");
    scanf("%d", &age);   // %d is format specifier for int

    printf("Enter your salary: ");
    scanf("%f", &salary); // %f is format specifier for float

    printf("You are %d years old and your salary is %.2f\n", age, salary);

    return 0;
}
```

CodeWithHary

Important Notes:

- Every data type has a **format specifier**. For example:
  - `%d` → int
  - `%f` → float
  - `%lf` → double
  - `%c` → char
  - `%s` → string (character array)
- The `&` symbol is used in `scanf` to provide the **address** of the variable where input should be stored.

Comments and Code Readability

Comments are non-executable parts of the code used to explain what the program is doing. They make the code easier to understand.

Types of comments in C:

- **Single-line comment**: Starts with `//`
- **Multi-line comment**: Written between `/* ... */`

Example:

```c
#include <stdio.h>

int main() {
    // This is a single-line comment
    int a = 10; /* This is a multi-line comment */
    printf("Value of a: %d\n", a);
    return 0;
}
```

Good programmers always use comments to describe tricky parts of code or to explain logic.

CodeWithHarry

Practice Set – Basics of C Programming

These exercises will help you practice variables, data types, input/output, and comments in C. Try solving them on your own before looking at the solutions.

Exercise 1: Print Student Details

Write a program that declares variables for a student's **age**, and **grade**, assigns values to them, and prints them.

Example Output:

```
Age: 18
Grade: A
```

Exercise 2: Simple Addition

Write a program that takes **two integers as input** from the user and prints their sum.

Example Input/Output:

```
Enter first number: 12
Enter second number: 8
Sum = 20
```

CodeWithHarry

Exercise 3: Swap Two Numbers

Write a program that swaps the values of two integer variables using a third temporary variable.

Example Output:

```
Before swap: a = 5, b = 10
After swap: a = 10, b = 5
```

Exercise 4: Circle Area Calculator

Write a program that asks the user to enter the radius of a circle and then calculates and prints its area. (Use the formula: Area = π * r * r, where π = 3.1416)

Example Input/Output:

```
Enter radius: 7
Area of circle = 153.94
```

Exercise 5: Comment Usage

Write a program that:

- Declares an integer variable `x` and assigns it a value.
- Prints the value of `x`.
- Uses comments to explain what each line is doing.

CodeWithHarry

Operators & Expressions

In this section, we will explore how to perform calculations and make decisions in C using **operators** and **expressions**. Operators are symbols that tell the compiler to perform specific mathematical or logical operations on variables and values.

Understanding operators is crucial because almost every line of code in C involves some kind of operation - whether it's arithmetic, comparison, or assignment.

Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations.

Operator	Description	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`5 - 3`	`2`
`*`	Multiplication	`5 * 3`	`15`
`/`	Division	`5 / 2`	`2` (integer division)
`%`	Modulus (remainder)	`5 % 2`	`1`

Example Program:

```c
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("Addition: %d\n", a + b);
    printf("Subtraction: %d\n", a - b);
    printf("Multiplication: %d\n", a * b);
```

CodeWithHarry

```c
    printf("Division: %d\n", a / b);
    printf("Remainder: %d\n", a % b);

    return 0;
}
```

Note: If both operands are integers, division `/` will give an integer result. For example, `10 / 3` gives `3`, not `3.33`. To get a floating-point result, at least one operand must be a float (e.g., `10.0 / 3`).

Relational and Logical Operators

Relational and logical operators are used to compare values and make decisions.

Relational Operators

These operators compare two values and return either `1` (true) or `0` (false).

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 3`	`0`
`!=`	Not equal to	`5 != 3`	`1`
`>`	Greater than	`5 > 3`	`1`
`<`	Less than	`5 < 3`	`0`
`>=`	Greater than or equal to	`5 >= 5`	`1`
`<=`	Less than or equal to	`5 <= 3`	`0`

CodeWithHarry

Logical Operators

Logical operators are used to combine multiple conditions.

They are used to build logic in C programs

Operator	Meaning	Example	Result
`&&`	Logical AND	`(a > b) && (a < 20)`	True if both are true
`||`	Logical OR	`(a > b) || (a < 0)`	True if at least one is true
`!`	Logical NOT	`!(a > b)`	True if condition is false

Example Program:

```c
#include <stdio.h>

int main() {
    int a = 5, b = 10;

    printf("a == b: %d\n", a == b);
    printf("a != b: %d\n", a != b);
    printf("a > b: %d\n", a > b);
    printf("a < b: %d\n", a < b);

    printf("(a < b) && (b < 20): %d\n", (a < b) && (b < 20));
    printf("(a > b) || (b < 20): %d\n", (a > b) || (b < 20));
    printf("!(a == b): %d\n", !(a == b));

    return 0;
}
```

CodeWithHarry

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Description	Example	Equivalent To
`=`	Assign	`a = 5`	`a = 5`

Operator	Description	Example	Equivalent To
`+=`	Add and assign	`a += 2`	`a = a + 2`
`-=`	Subtract and assign	`a -= 2`	`a = a - 2`
`*=`	Multiply and assign	`a *= 2`	`a = a * 2`
`/=`	Divide and assign	`a /= 2`	`a = a / 2`
`%=`	Modulus and assign	`a %= 2`	`a = a % 2`

Example Program:

```c
#include <stdio.h>

int main() {
    int a = 10;

    a += 5;
    printf("a after += 5: %d\n", a);

    a -= 3;
    printf("a after -= 3: %d\n", a);

    a *= 2;
    printf("a after *= 2: %d\n", a);

    a /= 4;
    printf("a after /= 4: %d\n", a);

    a %= 3;
    printf("a after %%= 3: %d\n", a);

    return 0;
}
```

CodeWithHarry

Increment and Decrement Operators

Increment (`++`) and decrement (`--`) operators are shortcuts for adding or subtracting 1 from a variable.

Operator	Meaning	Example	Result
`++`	Increment	`a++` or `++a`	Increase by 1
`--`	Decrement	`a--` or `--a`	Decrease by 1

There are two types of increment/decrement:

- **Pre-increment / Pre-decrement:** Changes the value before using it.
- **Post-increment / Post-decrement:** Uses the value first, then changes it.

Example Program:

```c
#include <stdio.h>

int main() {
    int a = 5;

    printf("Original a: %d\n", a);
    printf("Post-increment: %d\n", a++);
    printf("After post-increment: %d\n", a);

    a = 5; // reset a
    printf("Pre-increment: %d\n", ++a);
    printf("After pre-increment: %d\n", a);

    return 0;
}
```

CodeWithHarry

Type Casting

Type casting is converting a variable from one data type to another. This is often needed when performing operations between different data types.

Example Program:

```c
#include <stdio.h>

int main() {
    int a = 5, b = 2;
    float result;

    result = (float)a / b; // cast a to float before division
    printf("Result = %.2f\n", result);

    return 0;
}
```

Explanation: Without type casting, `5 / 2` would result in `2` (integer division). By converting `a` to float, we get a precise result: `2.50`.

Operator Precedence and Associativity

When multiple operators appear in an expression, **precedence** determines which operator is evaluated first, and **associativity** determines the direction (left to right or right to left).

Example of Precedence

In `3 + 5 * 2`, multiplication has higher precedence than addition, so the result is `3 + (5 * 2)` = `13`.

CodeWithHarry

Example of Associativity

For operators with the same precedence, C uses associativity. For example, in `a = b = c = 5`, the assignment operator `=` has **right-to-left** associativity, so it is evaluated as `a = (b = (c = 5))`.

Common Precedence Order (Highest to Lowest)

1. Parentheses `()`
2. Unary operators `++`, `--`, `!`
3. Multiplication, Division, Modulus `*`, `/`, `%`
4. Addition, Subtraction `+`, `-`
5. Relational operators `<`, `>`, `<=`, `>=`
6. Equality operators `==`, `!=`
7. Logical AND `&&`
8. Logical OR `||`
9. Assignment `=`, `+=`, `-=`

CodeWithHarry

Practice Set – Operators & Expressions

Exercise 1: Simple Arithmetic Calculator

Write a program that takes two integers as input and performs the following operations: addition, subtraction, multiplication, division, and modulus. Print the results in a clear format.

Example Input/Output:

```
Enter two numbers: 10 3
Sum = 13
Difference = 7
Product = 30
Quotient = 3
Remainder = 1
```

Exercise 2: Relational Operators

Write a program that takes two numbers from the user and prints 1 if the first number is greater than the second number (Otherwise it should print 0).

Example Input/Output:

```
Enter two numbers: 8 12
0
```

CodeWithHarry

Exercise 3: Logical Operators

Write a program that checks whether a person is eligible to vote. A person is eligible if their age is **18 or above** and **they are a citizen (use 1 for Yes, 0 for No)**. Print 1 for yes and 0 for no

Example Input/Output:

```
Enter age: 20
Are you a citizen (1 for Yes / 0 for No): 1
1
```

Exercise 4: Increment and Decrement

Write a program that demonstrates the difference between pre-increment (++x) and post-increment (x++) operations. Print the value of a variable before and after each operation.

Example Output:

```
x = 5
Pre-increment: 6
After pre-increment: 6
Post-increment: 6
After post-increment: 7
```

Exercise 5: Compound Assignment

Write a program that initializes an integer variable with some value and then applies the following compound assignment operations sequentially:

```
+=, -=, *=, /=, %=
```

Print the value of the variable after each operation.

CodeWithHarry

Exercise 6: Type Casting

Write a program that demonstrates **type casting** by dividing two integers and printing both the integer and floating-point results.

Example Output:

```
Integer division result: 2
Floating-point division result: 2.50
```

Exercise 7: Operator Precedence

Predict the output of this expression before running it:

```
int result = 10 + 20 * 3 / 2 - 5;
```

Then, write a program to verify your answer and explain why that result was produced.

CodeWithHarry

Control Flow & Decision Making

Hello everyone, Harry here. Now that you've learned how operators and expressions work in C, it's time to explore how we can make our programs *think* and *decide* what to do - this is where **control flow** comes in.

In real-world programming, you often need your program to make choices. For example:

- If a student's marks are above 90, print "Excellent".
- Otherwise, print "Keep trying".

Such decisions are handled using **conditional statements** in C.

The if Statement

The simplest decision-making statement in C is the `if` statement. It allows your program to execute a block of code **only if** a certain condition is true.

Syntax:

```
if (condition) {
    // code runs only if condition is true
}
```

Example:

```
int age = 20;

if (age >= 18) {
    printf("You are eligible to vote.");
}
```

CodeWithHarry

If `age` is 20, the condition `age >= 18` is true, so the message is printed.

The if-else Statement

The `if-else` structure lets you choose between **two possible actions**.

Syntax:

```
if (condition) {
    // runs if condition is true
} else {
    // runs if condition is false
}
```

Example:

```
int number = 10;

if (number % 2 == 0) {
    printf("Even number");
} else {
    printf("Odd number");
}
```

The if-else-if Ladder

Sometimes you need to test **multiple conditions**. In that case, you can use the `if-else-if` ladder.

Syntax:

```
if (condition1) {
    // executes if condition1 is true
```

CodeWithHary

```
} else if (condition2) {
    // executes if condition2 is true
} else {
    // executes if none of the above are true
}
```

Example:

```
int marks = 85;

if (marks >= 90) {
    printf("Grade A");
} else if (marks >= 75) {
    printf("Grade B");
} else if (marks >= 60) {
    printf("Grade C");
} else {
    printf("Grade D");
}
```

CodeWithHarry

Nested if Statements

You can also place one `if` statement inside another. This is called **nested if** and is used when one decision depends on another.

Example:

```
int age = 25;
char citizen = 'Y';

if (age >= 18) {
    if (citizen == 'Y') {
        printf("You are eligible to vote.");
    } else {
        printf("You are not a citizen, so you cannot vote.");
    }
```

```
} else {
    printf("You are not old enough to vote.");
}
```

---

switch Case Statement

When you have a long list of conditions checking for equality, using a `switch` statement is cleaner and faster.

Syntax:

```
switch (variable) {
    case value1:
        // code
        break;
    case value2:
        // code
        break;
    default:
        // code if no match found
}
```

Example:

```
int day = 3;

switch (day) {
    case 1:
        printf("Monday");
        break;
    case 2:
        printf("Tuesday");
        break;
    case 3:
        printf("Wednesday");
        break;
```

CodeWithHarry

```
    default:
        printf("Invalid day");
}
```

---

goto Statement (and Why to Avoid It)

The `goto` statement allows you to jump to a labeled section of code. While it might seem convenient, it makes code **hard to read and maintain**, and can cause **logical errors** if used carelessly.

Example:

```
#include <stdio.h>

int main() {
    int n = 1;

start:
    printf("%d ", n);
    n++;
    if (n <= 5)
        goto start;

    return 0;
}
```

CodeWithHarry

The above program prints numbers 1 to 5 using `goto`. However, it's better to use **loops**, which are cleaner and safer. You'll learn loops in the next section.

Summary

- `if` is used for simple decisions.
- `if-else` handles two-way decisions.

- `if-else-if` helps when there are multiple conditions.
- `switch` is cleaner when checking equality against many possible values.
- Avoid `goto` - prefer structured programming using loops and functions.

CodeWithHarry

Practice Set – Control Flow: Decision Making

This practice set will help you apply what you learned about **conditional statements** in C - how programs make decisions and choose between different actions.

Exercise 1: Check Even or Odd

Write a program that takes an integer from the user and prints whether it is even or odd.

Example Input/Output:

```
Enter a number: 7
7 is odd
```

Exercise 2: Find the Largest of Two Numbers

Write a program that takes two numbers as input and prints the larger one using an `if-else` statement.

Example Input/Output:

```
Enter two numbers: 12 9
12 is larger
```

CodeWithHarry

Exercise 3: Grading System

Write a program that takes a student's marks (0–100) and prints the grade as per the following rules:

- 90 and above → Grade A
- 75 to 89 → Grade B
- 60 to 74 → Grade C
- Below 60 → Grade D

Use an **if-else-if ladder**.

Example Input/Output:

```
Enter marks: 82
Grade B
```

Exercise 4: Leap Year Checker

Write a program that checks whether a given year is a leap year or not. A leap year is divisible by 4, but not by 100 unless it is also divisible by 400.

Example Input/Output:

```
Enter a year: 2024
2024 is a leap year
```

Exercise 5: Menu-Driven Calculator (Using switch)

Write a simple calculator program that uses a **switch statement** to perform operations based on user choice. Operations:

1. Addition
2. Subtraction
3. Multiplication

CodeWithHary

4. Division

Example Input/Output:

```
Enter first number: 8
Enter second number: 2
Enter operation (1-Add, 2-Sub, 3-Mul, 4-Div): 3
Result = 16
```

Exercise 6: Character Type Checker

Write a program that takes a single character as input and determines whether it is a **vowel**, **consonant**, **digit**, or **special character**. Use a combination of `if-else` and logical operators.

Example Input/Output:

```
Enter a character: a
Vowel
```

Exercise 7: Nested if – Eligibility for Admission

A student is eligible for admission if:

- Their math marks ≥ 60
- Their physics marks ≥ 50
- Their chemistry marks ≥ 40
- And the total marks ≥ 200

Use nested `if` statements to check eligibility.

Example Input/Output:

CodeWithHarry

```
Enter marks in Math, Physics, Chemistry: 70 55 50
Eligible for admission
```

Exercise 8: goto Example (Educational Only)

Write a program using `goto` to print numbers from 1 to 5. Then rewrite it using a **loop** (for or while) to show why loops are better.

Exercise 9: Simple Login System

Write a program that asks for a username and password (hardcode both), and prints “Login Successful” if both match, otherwise “Access Denied”. Use an `if-else` statement.

Exercise 10: Number Range Checker

Write a program that takes a number and prints whether it lies between **1–10**, **11–20**, or **greater than 20** using an `if-else-if` ladder.

Bonus Challenge: Grade Using switch

Recreate the **grading system (Exercise 3)** using a `switch` statement by dividing the marks by 10 and matching the integer value.

CodeWithHarry

Control Flow – Loops

Hello everyone, Harry here. In the previous section, we learned how to make decisions in our programs using `if`, `else`, and `switch`. Now, let's move on to something equally important i.e **loops**.

Loops allow you to **execute a block of code repeatedly** without writing it multiple times. For example, if you want to print "Hello, World!" ten times, you don't need ten `printf` statements, you can simply use a loop.

The `while` Loop

The `while` loop executes a block of code **as long as a given condition is true**. It is ideal when you don't know beforehand how many times you'll need to repeat something.

CodeWithHarry

Syntax:

```
while (condition) {
    // code to be executed
}
```

Example:

```
#include <stdio.h>

int main() {
    int i = 1;

    while (i <= 5) {
        printf("%d\n", i);
        i++;
```

```
    }

    return 0;
}
```

Explanation: The loop starts with `i = 1` and prints numbers from 1 to 5. It stops when `i` becomes greater than 5.

The `for` Loop

The `for` loop is the most commonly used loop in C. It's perfect when you know **exactly how many times** you want to repeat an action.

Syntax:

```
for (initialization; condition; increment/decrement) {
    // code to execute
}
```

Example:

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

Explanation:

- The loop initializes `i` as 1.
- It runs as long as `i <= 5`.
- After each iteration, `i` increases by 1.

CodeWithHarry

The `do-while` Loop

The `do-while` loop is similar to the `while` loop, except it **always executes the body at least once**, even if the condition is false.

Syntax:

```
do {
    // code to execute
} while (condition);
```

Example:

```c
#include <stdio.h>

int main() {
    int i = 1;

    do {
        printf("%d\n", i);
        i++;
    } while (i <= 5);

    return 0;
}
```

Explanation: Even if the condition becomes false initially, the loop body executes once before checking it again.

CodeWithHarry

Nested Loops

A **nested loop** means having one loop inside another. It is often used for tasks like printing patterns or working with multi-dimensional data.

Example:

```c
#include <stdio.h>

int main() {
    for (int i = 1; i <= 3; i++) {
        for (int j = 1; j <= 3; j++) {
            printf("(%d, %d) ", i, j);
        }
        printf("\n");
    }
    return 0;
}
```

Output:

```
(1, 1) (1, 2) (1, 3)
(2, 1) (2, 2) (2, 3)
(3, 1) (3, 2) (3, 3)
```

CodeWithHarry

---

break and continue

These two statements help you control the flow inside loops.

The `break` Statement

Used to **exit** a loop immediately, even if the condition is still true.

Example:

```c
#include <stdio.h>

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i == 5)
            break;
```

```
        printf("%d\n", i);
    }
    return 0;
}
```

Output:

```
1
2
3
4
```

The loop stops when `i` becomes 5.

The `continue` Statement

Used to **skip** the current iteration and move to the next one.

Example:

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        if (i == 3)
            continue;
        printf("%d\n", i);
    }
    return 0;
}
```

Output:

```
1
2
```

CodeWithHarry

```
4
5
```

When `i` equals 3, the loop skips printing and continues with the next iteration.

Summary

- `while` → Use when you don't know the exact number of repetitions.
- `for` → Use when the number of iterations is known.
- `do-while` → Executes at least once.
- Nested loops → Used for multi-level repetitions or patterns.
- `break` → Stops the loop immediately.
- `continue` → Skips the current iteration.

CodeWithHarry

Practice Set – Control Flow and Loops

In this practice set, you'll work on problems that use different types of loops in C: `while`, `for`, and `do-while`. You'll also practice nested loops, and understand when to use `break` and `continue`.

Exercise 1: Print 1 to 10

Write a program using a `for` loop to print numbers from 1 to 10.

Example Output:

```
1 2 3 4 5 6 7 8 9 10
```

Exercise 2: Sum of First N Natural Numbers

Write a program using a `while` loop that calculates the sum of the first **N** natural numbers entered by the user.

Example Input/Output:

```
Enter a number: 5
Sum = 15
```

Exercise 3: Multiplication Table

Write a program that takes an integer as input and prints its multiplication table up to 10.

CodeWithHarry

Example Input/Output:

```
Enter a number: 7
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Exercise 4: Reverse Counting

Write a program that prints numbers from 10 down to 1 using a `while` loop.

Example Output:

```
10 9 8 7 6 5 4 3 2 1
```

Exercise 5: Factorial of a Number

Write a program that takes a number as input and prints its factorial. Use a `for` loop.

Example Input/Output:

```
Enter a number: 5
Factorial = 120
```

Exercise 6: do-while Loop Example

Write a program that keeps asking the user to enter a positive number until they enter 0. Use a `do-while` loop.

Example Input/Output:

CodeWithHarry

```
Enter a number: 4
Enter a number: 9
Enter a number: 0
Program ended
```

Exercise 7: Sum of Even Numbers (Using continue)

Write a program that prints the sum of all even numbers between 1 and 20. Use `continue` to skip odd numbers.

Example Output:

```
Sum of even numbers between 1 and 20 = 110
```

Exercise 8: Stop at First Negative (Using break)

Write a program that takes numbers as input continuously. Stop reading and exit the loop as soon as a negative number is entered.

Example Input/Output:

```
Enter number: 5
Enter number: 3
Enter number: -1
Loop terminated
```

Exercise 9: Nested Loop – Pattern Printing (Square Pattern)

Write a program to print a square pattern of stars of size `n` using nested loops.

Example Input/Output:

CodeWithHarry

```
Enter n: 4
* * * *
* * * *
* * * *
* * * *
```

Exercise 10: Nested Loop – Right Triangle Pattern

Write a program to print the following right triangle pattern using nested loops.

Example Input/Output:

```
Enter n: 5
*
* *
* * *
* * * *
* * * * *
```

Bonus Challenge: Multiplication Table Grid

Write a program that prints a multiplication table from 1 to 10 in a grid format using nested `for` loops.

Example Output (partial):

```
1  2  3  4  5  6  7  8  9 10
2  4  6  8 10 12 14 16 18 20
3  6  9 12 15 18 21 24 27 30
...
```

CodeWithHarry

Functions in C

Hello everyone, Harry here. In this part of the course, we'll talk about one of the most important building blocks of C programming i.e **functions**. Functions help you organize and reuse your code efficiently. Instead of writing the same code again and again, you can define it once inside a function and call it whenever you need it.

What are Functions?

A function is a block of code designed to perform a specific task. Every C program has at least one function which is `main()`, where the execution starts. You can also create your own functions to make your code modular and readable.

Syntax:

```
return_type function_name(parameters) {
    // body of the function
}
```

Example:

```
#include <stdio.h>

void greet() {
    printf("Hello, World!");
}

int main() {
    greet(); // function call
    return 0;
}
```

CodeWithHarry

Here, `greet()` is a user-defined function that prints a message. The `main()` function calls it.

Function Definition & Declaration

Functions in C can be defined in two ways:

1. **Before the main function** (direct definition)
2. **After the main function** (requires declaration)

A **function declaration** (or prototype) tells the compiler that a function exists before it is used.

Example:

```c
#include <stdio.h>

// Function declaration
void greet();

int main() {
    greet(); // function call
    return 0;
}

// Function definition
void greet() {
    printf("Hello from a function!");
}
```

CodeWithHarry

Parameters & Return Values

Functions can accept input values (parameters) and can also return a result.

Example: Function with Parameters and Return Value

```c
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int sum = add(5, 3);
    printf("Sum = %d", sum);
    return 0;
}
```

Explanation:

- `int add(int a, int b)` defines a function that takes two integers and returns an integer.
- The `return` statement sends the result back to the caller.

Call by Value vs Call by Reference

In C, function arguments can be passed in two ways.

Call by Value

A **copy** of the variable's value is passed to the function. Changes made inside the function do not affect the original variable.

Example:

```c
#include <stdio.h>

void change(int x) {
    x = 10;
}

int main() {
```

CodeWithHarry

```
    int a = 5;
    change(a);
    printf("%d", a); // Output: 5
    return 0;
}
```

Call by Reference

The **address** of the variable is passed using pointers. Changes inside the function affect the original variable.

Example:

```
#include <stdio.h>

void change(int *x) {
    *x = 10;
}

int main() {
    int a = 5;
    change(&a);
    printf("%d", a); // Output: 10
    return 0;
}
```

Recursion in C

Recursion is when a function calls itself to solve a smaller version of a problem. Every recursive function must have a **base case** to prevent infinite calls.

Example: Factorial using Recursion

```
#include <stdio.h>

int factorial(int n) {
```

CodeWithHary

```
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;
    printf("Factorial of %d = %d", num, factorial(num));
    return 0;
}
```

How it works:

- `factorial(5)` calls `factorial(4)`, then `factorial(3)`, and so on.
- When `n` becomes 0, the recursion stops and returns 1.

Functions are the backbone of modular programming. They help break down complex problems into smaller, manageable tasks. In the next part, you'll get practice exercises to strengthen your understanding of how functions work in C.

CodeWithHarry

Practice Set on Functions in C

Here are some exercises to help you practice and strengthen your understanding of functions in C.

Exercise 1: Write Your First Function

Write a function named `greet()` that prints "Hello, welcome to C programming!". Call this function from the `main()` function.

Expected Output:

```
Hello, welcome to C programming!
```

Exercise 2: Add Two Numbers Using a Function

Write a function `add(int a, int b)` that takes two integers as arguments and returns their sum. Call the function from `main()` and display the result.

Example Input/Output:

```
Enter two numbers: 5 7
Sum = 12
```

Exercise 3: Find Maximum of Two Numbers

Write a function `max(int x, int y)` that returns the greater of two numbers. Use this function inside `main()` to find and print the maximum number.

CodeWithHarry

Example Input/Output:

```
Enter two numbers: 10 25
Maximum = 25
```

Exercise 4: Factorial Using Function

Write a function `factorial(int n)` that returns the factorial of `n`. Use a `for` loop inside the function.

Example Input/Output:

```
Enter a number: 5
Factorial = 120
```

Exercise 5: Demonstrate Call by Value

Write a program to show that changes made to a variable inside a function using **call by value** do not affect the original variable.

Example Output:

```
Before function call: a = 10
Inside function: a = 20
After function call: a = 10
```

Exercise 6: Demonstrate Call by Reference

Write a function `swap(int *x, int *y)` that swaps the values of two integers using pointers.

Example Input/Output:

CodeWithHarry

```
Before swap: x = 10, y = 20
After swap: x = 20, y = 10
```

Exercise 7: Recursive Function for Factorial

Write a recursive version of the factorial program using a function `factorial(int n)`.

Example Input/Output:

```
Enter a number: 5
Factorial = 120
```

Exercise 8: Recursive Fibonacci Sequence

Write a recursive function `fibonacci(int n)` that prints the first `n` terms of the Fibonacci series.

Example Input/Output:

```
Enter number of terms: 6
Fibonacci Series: 0 1 1 2 3 5
```

Exercise 9: Power of a Number (Using Recursion)

Write a recursive function `power(int base, int exp)` to calculate `base^exp`.

Example Input/Output:

CodeWithHarry

```
Enter base: 2
Enter exponent: 5
Result = 32
```

Exercise 10: Sum of Digits Using Function

Write a function `sumOfDigits(int n)` that returns the sum of digits of a given number.

Example Input/Output:

```
Enter a number: 1234
Sum of digits = 10
```

CodeWithHarry

Arrays in C

Arrays in C are collections of elements of the same data type stored in contiguous memory locations. They allow you to store and access multiple values using a single variable name.

Declaring and Initializing Arrays

You can declare an array like this:

```
int numbers[5];
```

You can also initialize it at the time of declaration:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

If you initialize all elements, you can omit the size:

```
int numbers[] = {1, 2, 3, 4, 5};
```

Accessing Array Elements

Array elements are accessed using indices starting from 0.

```
#include <stdio.h>

int main() {
    int marks[3] = {85, 90, 78};
```

CodeWithHarry

```
    printf("First mark = %d\n", marks[0]);
    printf("Second mark = %d\n", marks[1]);
    printf("Third mark = %d\n", marks[2]);
    return 0;
}
```

Taking Input for Arrays

You can use loops to take user input for array elements.

```
#include <stdio.h>

int main() {
    int arr[5];
    for (int i = 0; i < 5; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    printf("You entered: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Traversing an Array

You can traverse an array using a `for` loop to perform operations like summing all elements.

```
#include <stdio.h>
```

CodeWithHarry

```c
int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int sum = 0;
    for (int i = 0; i < 5; i++) {
        sum += arr[i];
    }
    printf("Sum = %d", sum);
    return 0;
}
```

Modifying Array Elements

You can change any array element using its index.

```c
int arr[3] = {2, 4, 6};
arr[1] = 10;
printf("%d", arr[1]); // Output: 10
```

Arrays and Functions

Arrays can be passed to functions as arguments.

```c
#include <stdio.h>

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    printArray(numbers, 5);
```

CodeWithHarry

```
    return 0;
}
```

Multidimensional Arrays

C also supports arrays with more than one dimension, like 2D arrays (matrices).

```c
#include <stdio.h>

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Memory Layout of Arrays

Arrays store elements in contiguous memory locations, which means all elements are stored next to each other. This allows fast access to elements using their index but makes resizing difficult.

CodeWithHarry

Common Mistakes with Arrays

1. Accessing elements beyond the array size (out-of-bounds).
2. Forgetting to initialize arrays before use.
3. Using incorrect array sizes when passing to functions.

Example Program: Find Maximum Element

```c
#include <stdio.h>

int main() {
    int arr[5] = {12, 45, 23, 67, 34};
    int max = arr[0];

    for (int i = 1; i < 5; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }

    printf("Maximum element = %d", max);
    return 0;
}
```

CodeWithHarry

Practice Set on Arrays in C

Exercise 1: Store and Print 5 Integers

Write a program to take 5 integers as input from the user and print them back on the screen.

Example Input/Output:

```
Enter 5 numbers: 1 2 3 4 5
You entered: 1 2 3 4 5
```

Exercise 2: Sum of Array Elements

Write a program to calculate the sum of all elements in an array.

Example Input/Output:

```
Enter 5 numbers: 10 20 30 40 50
Sum = 150
```

Exercise 3: Find Maximum and Minimum Element

Write a program to find the largest and smallest numbers in an array.

Example Input/Output:

CodeWithHary

```
Enter 5 numbers: 3 8 2 9 6
Maximum = 9
Minimum = 2
```

Exercise 4: Reverse an Array

Write a program to reverse the elements of an array.

Example Input/Output:

```
Original array: 1 2 3 4 5
Reversed array: 5 4 3 2 1
```

Exercise 5: Count Even and Odd Numbers

Write a program that counts how many even and odd numbers are in an array.

Example Input/Output:

```
Enter 6 numbers: 2 5 7 8 10 11
Even = 3
Odd = 3
```

Exercise 6: Search for an Element

Write a program to search for a given number in an array.

Example Input/Output:

```
Enter 5 numbers: 4 7 9 2 5
Enter number to search: 9
9 found at index 2
```

CodeWithHary

Exercise 7: Copy Array Elements

Write a program to copy all elements of one array into another array.

Example Input/Output:

```
Original array: 1 2 3 4 5
Copied array: 1 2 3 4 5
```

Exercise 8: Sum of Elements at Even and Odd Indices

Write a program to find the sum of array elements present at even and odd indices separately.

Example Input/Output:

```
Array: 10 20 30 40 50
Sum of even indices = 90
Sum of odd indices = 60
```

Exercise 9: Second Largest Element

Write a program to find the second largest number in an array.

Example Input/Output:

```
Array: 12 35 1 10 34 1
Second largest = 34
```

CodeWithHarry

Exercise 10: Sum of Two Arrays

Write a program that takes two arrays of the same size and stores their element-wise sum in a third array.

Example Input/Output:

```
Array 1: 1 2 3 4 5
Array 2: 5 4 3 2 1
Sum Array: 6 6 6 6 6
```

CodeWithHarry

Strings in C

Strings in C are arrays of characters that end with a null character (`'\0'`). They are widely used for storing and manipulating text.

Declaring and Initializing Strings

A string can be declared in two ways:

```
char str1[20] = "Hello";
char str2[] = {'H', 'e', 'l', 'l', 'o', '\0'};
```

Both represent the same string `"Hello"` . The first method is more convenient and commonly used.

Input and Output with Strings

We can use `printf()` and `scanf()` to print and take input for strings.

```
#include <stdio.h>

int main() {
    char name[50];
    printf("Enter your name: ");
    scanf("%s", name); // Note: scanf stops reading at the first space
    printf("Hello, %s!", name);
    return 0;
}
```

CodeWithHarry

For multi-word input, use `gets()` (deprecated) or `fgets()` instead of `scanf()`.

```
fgets(name, sizeof(name), stdin);
```

Common String Functions

C provides many functions for string manipulation through the `<string.h>` header file.

```
#include <string.h>
```

Some common functions are:

Function	Description
`strlen(str)`	Returns the length of the string
`strcpy(dest, src)`	Copies one string into another
`strcat(dest, src)`	Concatenates two strings
`strcmp(str1, str2)`	Compares two strings
`strrev(str)`	Reverses a string (not standard C, available in some compilers)

Example:

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[50] = "Hello";
```

CodeWithHarry

```c
    char str2[50] = "World";

    printf("Length of str1: %lu\n", strlen(str1));

    strcat(str1, str2);
    printf("Concatenated String: %s\n", str1);

    strcpy(str2, "C Language");
    printf("Copied String: %s\n", str2);

    printf("Comparison Result: %d\n", strcmp("abc", "abd"));
    return 0;
}
```

Traversing a String

You can use a loop to access each character of a string.

```c
#include <stdio.h>

int main() {
    char str[] = "CodeWithHarry";
    for (int i = 0; str[i] != '\0'; i++) {
        printf("%c ", str[i]);
    }
    return 0;
}
```

CodeWithHarry

String and Character Arrays

Remember, strings are just character arrays. You can modify individual characters just like array elements.

```c
char word[] = "Cat";
word[0] = 'B';
printf("%s", word); // Output: Bat
```

Null Character Importance

All strings in C must end with a null character (`'\0'`). Without it, functions like `printf()` won't know where the string ends, which may cause unexpected results.

```c
char name[5] = {'H', 'a', 'r', 'r', 'y'}; // No '\0'
printf("%s", name); // Undefined behavior
```

Always include the null terminator or use string literals.

String vs Character

A character is a single symbol enclosed in single quotes (`'A'`), while a string is a sequence of characters enclosed in double quotes (`"A"`).

String Example Program

```c
#include <stdio.h>
#include <string.h>

int main() {
    char name[50];
    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);
    printf("Your name is: %s", name);
    printf("Length: %lu\n", strlen(name));
```

CodeWithHarry

```
    return 0;
}
```

CodeWithHarry

Practice Set on Strings in C

Exercise 1: Print a String

Write a program to take a string as input from the user and print it.

Example Input/Output:

```
Enter a string: CodeWithHarry
You entered: CodeWithHarry
```

Exercise 2: Find Length of a String

Write a program to find the length of a string without using the `strlen()` function.

Example Input/Output:

```
Enter a string: Hello
Length = 5
```

Exercise 3: Copy One String to Another

Write a program to copy one string into another without using the `strcpy()` function.

Example Input/Output:

CodeWithHarry

```
Enter a string: CProgramming
Copied string: CProgramming
```

Exercise 4: Compare Two Strings

Write a program to compare two strings without using the `strcmp()` function.

Example Input/Output:

```
Enter first string: apple
Enter second string: apple
Strings are equal
```

Exercise 5: Concatenate Two Strings

Write a program to concatenate two strings without using the `strcat()` function.

Example Input/Output:

```
Enter first string: Hello
Enter second string: World
Concatenated string: HelloWorld
```

Exercise 6: Count Vowels and Consonants

Write a program to count the number of vowels and consonants in a string.

Example Input/Output:

CodeWithHarry

```
Enter a string: Programming
Vowels = 3
Consonants = 8
```

Exercise 7: Reverse a String

Write a program to reverse a string without using the `strrev()` function.

Example Input/Output:

```
Enter a string: Harry
Reversed string: yrraH
```

Exercise 8: Palindrome Check

Write a program to check whether a given string is a palindrome or not.

Example Input/Output:

```
Enter a string: madam
madam is a palindrome
```

Exercise 9: Count Words in a Sentence

Write a program to count the number of words in a given sentence.

Example Input/Output:

```
Enter a sentence: I love coding in C
Word count = 5
```

CodeWithHarry

Exercise 10: Convert to Uppercase and Lowercase

Write a program to convert a string into uppercase and lowercase without using string library functions.

Example Input/Output:

```
Enter a string: CodeWithHarry
Uppercase: CODEWITHHARRY
Lowercase: codewithharry
```

CodeWithHarry

Pointers in C

Pointers are variables that store the memory address of another variable. They are one of the most powerful features of the C language, allowing direct access and manipulation of memory.

Declaring and Initializing Pointers

A pointer is declared using the `*` symbol.

```
int *ptr;
int a = 10;
ptr = &a; // Assigning address of a to ptr
```

Here, `ptr` holds the address of variable `a`.

Accessing Values Using Pointers

You can access the value stored at a pointer's address using the dereference operator `*`.

```
#include <stdio.h>

int main() {
    int a = 10;
    int *ptr = &a;

    printf("Address of a: %p\n", ptr);
    printf("Value of a: %d\n", *ptr);
```

CodeWithHarry

```
    return 0;
}
```

Pointer to Pointer

A pointer can also store the address of another pointer.

```c
#include <stdio.h>

int main() {
    int x = 5;
    int *p = &x;
    int **pp = &p;

    printf("Value of x = %d\n", **pp);
    return 0;
}
```

Pointers and Arrays

In C, the name of an array acts as a pointer to its first element.

```c
#include <stdio.h>

int main() {
    int arr[3] = {10, 20, 30};
    int *ptr = arr;

    for (int i = 0; i < 3; i++) {
        printf("%d ", *(ptr + i));
    }
    return 0;
}
```

CodeWithHarry

Pointer Arithmetic

Pointers can be incremented or decremented. When you increment a pointer, it moves to the next memory location based on the data type size.

```c
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3};
    int *ptr = arr;

    printf("%d ", *ptr);     // 1
    printf("%d ", *(ptr+1)); // 2
    printf("%d ", *(ptr+2)); // 3
    return 0;
}
```

Passing Pointers to Functions

Pointers are used to pass variables by reference to functions, allowing functions to modify the original data.

```c
#include <stdio.h>

void changeValue(int *n) {
    *n = 20;
}

int main() {
    int a = 10;
    changeValue(&a);
    printf("a = %d", a);
    return 0;
}
```

CodeWithHarry

Pointers and Functions

Pointers can also be used to return multiple values from functions using call by reference.

```c
#include <stdio.h>

void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}

int main() {
    int a = 5, b = 10;
    swap(&a, &b);
    printf("a = %d, b = %d", a, b);
    return 0;
}
```

Pointer to String

A string can be represented using a character pointer.

```c
#include <stdio.h>

int main() {
    char *name = "Harry";
    printf("%s", name);
    return 0;
}
```

CodeWithHarry

NULL Pointer

A pointer that does not point to any valid memory address is called a `NULL` pointer. It is used for safety to avoid undefined behavior.

```
int *ptr = NULL;
```

Dangling Pointer

A dangling pointer occurs when the memory it points to is freed or goes out of scope. Always set pointers to `NULL` after freeing memory to avoid this problem.

Pointer Best Practices

1. Always initialize pointers before use.
2. Avoid accessing invalid memory locations.
3. Use `NULL` to check if a pointer is valid.
4. Free dynamically allocated memory before program termination.

Example Program: Sum Using Pointers

```
#include <stdio.h>

int main() {
    int a = 10, b = 20, sum;
    int *p1 = &a, *p2 = &b;

    sum = *p1 + *p2;
    printf("Sum = %d", sum);
```

CodeWithHarry

```
    return 0;
}
```

CodeWithHarry

Practice Set on Pointers in C

Exercise 1: Pointer Basics

Write a program to print the address and value of a variable using a pointer.

Example Output:

```
Address of variable: 0x7ffca4b7a9b4
Value of variable: 10
```

Exercise 2: Pointer to Pointer

Write a program to demonstrate a pointer to a pointer.

Example Output:

```
Value of x = 5
Value accessed using pointer to pointer = 5
```

Exercise 3: Add Two Numbers Using Pointers

Write a program to add two numbers using pointers.

Example Input/Output:

```
Enter two numbers: 10 20
Sum = 30
```

CodeWithHarry

Exercise 4: Swap Two Numbers Using Pointers

Write a program to swap two numbers using pointers.

Example Input/Output:

```
Before swap: a = 5, b = 10
After swap: a = 10, b = 5
```

Exercise 5: Array Traversal Using Pointers

Write a program to print all elements of an array using pointers.

Example Input/Output:

```
Array elements: 10 20 30 40 50
```

Exercise 6: Find Maximum Element in Array Using Pointers

Write a program to find the maximum element in an array using pointers.

Example Input/Output:

```
Array: 12 45 23 67 34
Maximum element = 67
```

Exercise 7: Count Vowels Using Pointer

Write a program to count vowels in a string using pointers.

Example Input/Output:

CodeWithHarry

```
Enter a string: Harry
Vowel count = 1
```

Exercise 8: Change Value Using Function and Pointer

Write a function that takes an integer pointer as a parameter and changes the value of the variable it points to.

Example Output:

```
Before function call: x = 10
After function call: x = 50
```

Exercise 9: Pointer Arithmetic

Write a program that demonstrates pointer arithmetic using an integer array. Print the address before and after incrementing the pointer.

Example Output:

```
Address before increment: 6422288
Address after increment: 6422292
```

Exercise 10: Sum of Array Elements Using Pointer

Write a program to calculate the sum of elements of an array using pointers.

Example Input/Output:

```
Array: 1 2 3 4 5
Sum = 15
```

CodeWithHarry

Dynamic Memory Allocation in C

Dynamic Memory Allocation allows programs to request memory during runtime instead of compile time. This provides flexibility when the size of data is not known in advance.

Static vs Dynamic Memory Allocation

In static memory allocation, the memory size is fixed at compile time using arrays or variables. In dynamic allocation, memory is requested during program execution using special functions from the `stdlib.h` library.

malloc()

The `malloc()` function allocates a specified number of bytes and returns a pointer to the first byte of the block. It does not initialize the memory.

```
int *ptr = (int*) malloc(5 * sizeof(int));
```

calloc()

The `calloc()` function allocates multiple blocks of memory and initializes them to zero.

```
int *ptr = (int*) calloc(5, sizeof(int));
```

realloc()

The `realloc()` function is used to resize a previously allocated memory block.

CodeWithHarry

```
ptr = (int*) realloc(ptr, 10 * sizeof(int));
```

free()

The `free()` function deallocates memory that was previously allocated dynamically. It is important to free unused memory to avoid memory leaks.

```
free(ptr);
```

Common Mistakes

- Forgetting to free memory.
- Accessing memory after it has been freed.
- Using uninitialized pointers.
- Not checking if `malloc()` or `calloc()` returned `NULL`.

CodeWithHarry

Practice Set on Dynamic Memory Allocation in C

Question 1

Write a program to dynamically allocate memory for an array of 5 integers. Take input from the user and print the array elements.

Question 2

Use `calloc()` to allocate memory for 10 float values. Initialize them with random values and print their sum.

Question 3

Write a program to dynamically allocate memory for `n` integers, where `n` is entered by the user. Then, use `realloc()` to double the size and fill the new elements with user input.

Question 4

Demonstrate memory deallocation using `free()`. Allocate memory for an array, print values, free it, and then reallocate it again.

Question 5

Write a program that uses dynamic memory allocation to store names of students using an array of character pointers. Take input for names and display them.

Question 6

Use `malloc()` to create a dynamic 2D array (matrix) and take its elements as input. Print the matrix.

CodeWithHary

Question 7

Write a program to check whether `malloc()` or `calloc()` failed to allocate memory and handle the error gracefully by printing a message.

Question 8

Create a program to dynamically store marks of students using structures and pointers. Take inputs for names and marks, then display the results.

Question 9

Write a program that allocates memory for an array of integers, fills it with even numbers up to `n`, and then resizes it to hold odd numbers up to `n` as well using `realloc()`.

Question 10

Demonstrate the difference between `malloc()` and `calloc()` by showing their default initialization behavior in a program.

CodeWithHarry

Structures and Unions in C

What are Structures?

Structures in C are user-defined data types that allow grouping variables of different types under one name. They are used to represent a record.

Example:

```
struct Student {
    int id;
    char name[50];
    float marks;
};
```

Defining and Using Structures

You can declare a structure variable at the time of definition or later in the program.

Example:

```
struct Student s1 = {1, "Harry", 95.5};
printf("Name: %s, Marks: %.2f", s1.name, s1.marks);
```

Nested Structures

A structure can contain another structure as a member.

Example:

```
struct Address {
    char city[30];
    int pincode;
```

CodeWithHarry

```
};

struct Student {
    char name[30];
    struct Address addr;
};
```

Arrays of Structures

You can create an array of structures to store data for multiple entities.

Example:

```
struct Student students[3];
```

Pointers to Structures

Pointers can be used to access structure members efficiently using the arrow operator (`->`).

Example:

```
struct Student *ptr = &s1;
printf("%d", ptr->id);
```

Structures and Functions

You can pass structures to functions either by value or by reference.

Example:

```
void display(struct Student s);
void display(struct Student *s);
```

CodeWithHarry

Unions in C

Unions are similar to structures but share the same memory location for all members. Only one member can hold a value at a time.

Example:

```
union Data {
    int i;
    float f;
    char str[20];
};
```

Difference Between Structures and Unions

Feature	Structure	Union
Memory	Allocates memory for all members	Allocates memory equal to largest member
Access	All members can be accessed at once	Only one member can be used at a time
Use Case	When storing different data types together	When storing data that can take multiple forms

typedef Keyword

The `typedef` keyword allows you to create an alias for a data type, making your code more readable. **Example:** Here is an example of typedef with structures:

```
typedef struct {
    int id;
    char name[50];
    float marks;
} Student;
Student s1 = {1, "Harry", 95.5};
printf("Name: %s, Marks: %.2f", s1.name, s1.marks);
```

CodeWithHarry

Here is an example of typedef with unions:

```
typedef union {
    int i;
    float f;
    char str[20];
} Data;
Data d1;
d1.i = 10;
printf("Integer: %d", d1.i);
```

Hence, typedef simplifies the syntax and improves code clarity when working with structures and unions.

CodeWithHarry

Practice Set on Structures and Unions in C

Question 1

Create a structure `Book` with members `title`, `author`, and `price`. Write a program to input details for 3 books and display them.

Question 2

Write a program to calculate and display the average marks of 5 students using a structure `Student` with members `name` and `marks`.

Question 3

Define a structure `Employee` with members `id`, `name`, and `salary`. Write a function `display()` that takes a structure as an argument and prints its details.

Question 4

Write a program using a structure `Point` to store x and y coordinates. Calculate the distance between two points entered by the user.

Question 5

Create a structure `Date` with members `day`, `month`, and `year`. Write a program to check if two given dates are the same or not.

CodeWithHarry

Question 6

Write a program to store details of 10 employees using an array of structures and display those with a salary greater than 50,000.

Question 7

Demonstrate the use of pointers to structures by creating a structure `Car` with members `name` and `price`. Input and display the values using a pointer.

Question 8

Write a program that uses nested structures to store and display information about a student, including name, roll number, and address (city and pincode).

Question 9

Create a union `Data` that can store an integer, a float, or a string. Assign values to each member one by one and print them to demonstrate how only one value is retained at a time.

Question 10

Write a program that compares the memory size of a structure and a union containing an `int`, `float`, and `char` array of size 20. Display the sizes using `sizeof()`.

CodeWithHarry

File Handling in C

Introduction to File Handling

File handling in C allows programs to store and retrieve data permanently from secondary storage like a hard disk. C provides functions in the standard I/O library (`stdio.h`) to work with files easily.

File Operations in C

There are four basic operations that can be performed on files:

1. Creating a file
2. Opening a file
3. Reading from and writing to a file
4. Closing a file

File Pointers

C uses a special pointer of type `FILE*` to handle files. It is declared as:

```
FILE *ptr;
```

Opening and Closing a File

You can open a file using the `fopen()` function and close it using `fclose()` .

Example:

CodeWithHarry

```c
FILE *ptr;
ptr = fopen("example.txt", "w");  // Open file in write mode
if (ptr == NULL) {
    printf("File not found!");
    return 1;
}
fclose(ptr);
```

File Modes in C

Mode	Description
`"r"`	Open for reading (file must exist)
`"w"`	Open for writing (creates a new file or overwrites existing)
`"a"`	Open for appending (adds to the end of file)
`"r+"`	Open for reading and writing
`"w+"`	Create for reading and writing (overwrites existing)
`"a+"`	Open for reading and appending

Writing to a File

You can write data using `fprintf()` or `fputc()`.

Example:

```c
FILE *ptr = fopen("data.txt", "w");
fprintf(ptr, "Hello, C Programming!");
fclose(ptr);
```

CodeWithHarry

Reading from a File

You can read data using `fscanf()`, `fgetc()`, or `fgets()`.

Example:

```
FILE *ptr = fopen("data.txt", "r");
char str[100];
fgets(str, 100, ptr);
printf("File content: %s", str);
fclose(ptr);
```

Checking End of File

Use the `feof()` function to detect the end of a file while reading.

Example:

```
while (!feof(ptr)) {
    ch = fgetc(ptr);
    printf("%c", ch);
}
```

Binary File Operations

C also supports reading and writing binary files using `fread()` and `fwrite()`.

Example:

```
struct Student {
    char name[50];
    int marks;
};

struct Student s = {"Harry", 90};
```

CodeWithHarry

```
FILE *ptr = fopen("student.dat", "wb");
fwrite(&s, sizeof(s), 1, ptr);
fclose(ptr);
```

Error Handling in File Operations

Always check if the file pointer is `NULL` before performing file operations to avoid crashes.

Example:

```
if (ptr == NULL) {
    printf("Error opening file!");
    exit(1);
}
```

Closing a File

After performing all file operations, always close the file using:

```
fclose(ptr);
```

CodeWithHarry

Practice Set on File Handling in C

Question 1

Write a program to create a file named `data.txt` and write your name and age into it using `fprintf()`.

Question 2

Create a program that reads the content of `data.txt` and displays it on the screen using `fgetc()`.

Question 3

Write a program to copy the contents of one file into another file. Take source and destination file names from the user.

Question 4

Write a program that counts the number of characters, words, and lines in a text file.

Question 5

Create a file `numbers.txt` containing a list of integers. Write a program to read all numbers and calculate their sum.

CodeWithHarry

Question 6

Write a program to append a new line to an existing file without overwriting its content.

Question 7

Create a structure `Student` with members `name` and `marks`. Write multiple student records to a binary file using `fwrite()` and then read them back using `fread()`.

Question 8

Write a program that reads a text file and converts all lowercase letters to uppercase in another file.

Question 9

Write a program that takes a file name as input and checks whether it exists or not. Display a suitable message accordingly.

Question 10

Write a program that reads from a file character by character until the end of file using `feof()` and prints all characters to the console.

CodeWithHarry